Digital Fees Register

Technical Specification — v1.0 (build-ready)

Single source for §3 Data Models and §4 Validation & Business Logic — supersedes the standalone addendum.

Read this first. Two authoritative documents back the v1 build: this technical spec (schema, validation, scope) and Mockup_User_Functionality.md (interaction-level source of truth). The HTML mockup is the visual reference. The earlier Tech_Spec_Addendum_v0.9.md file has been folded into §3 and §4 below and retired.

1. Stack

Everything is mainstream and free at v1 scale. The whole toolchain costs ₹0.

Layer	Choice	Why
Framework	Next.js 15 + TypeScript	Mainstream in 2026, deploys to Vercel in one click.
Styling	Tailwind + shadcn/ui	Components copy into the repo — no runtime UI library to ship.
Cloud DB + auth	Supabase	Postgres, Auth, and Realtime in one dashboard. Free tier.
Local cache	Dexie on IndexedDB	Writes survive power cuts and weak internet, then replay when online.
Hosting	Vercel Hobby	Auto-deploys from GitHub, preview URL per pull request. Free.
Code	GitHub private repo	Version control and off-machine backup. Free.
Analytics (optional)	PostHog	Tracks the per-entry speed target. Free tier; skip if cutting scope.

2. Architecture

Two layers: the browser the Principal works in, and Supabase in the cloud. Vercel serves the Next.js app to the browser; everything the Principal types lands first in the browser's own storage, then syncs up to Supabase.

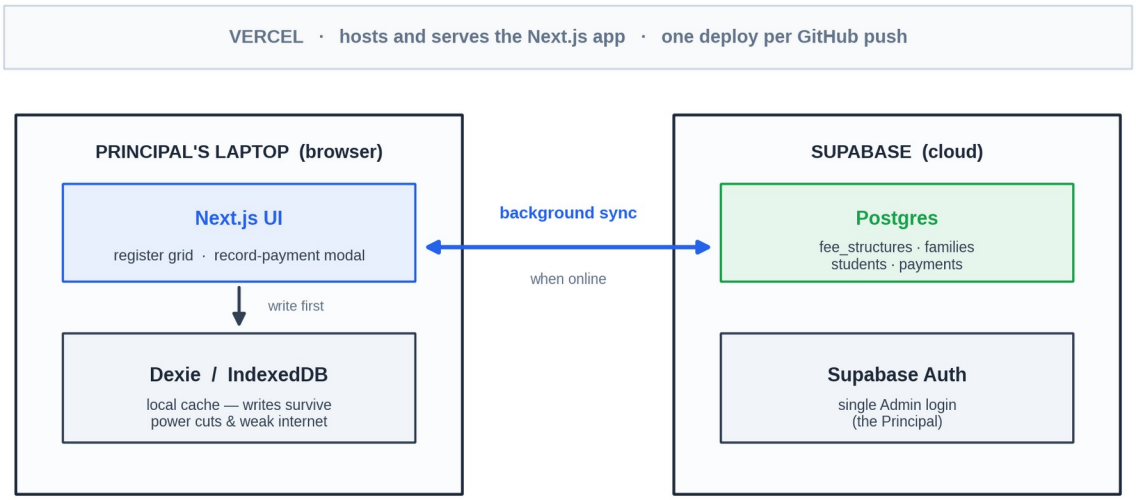

Figure 2.1 — Architecture layers (Next.js + Supabase + Dexie/IndexedDB)

Recording a payment

The write path is deliberately short — the Principal never waits on the network.

Figure 2.2 — Write path: tap cell → Dexie (instant) → background sync to Supabase

Reading, and staying online-optional

Reads come from the local IndexedDB cache after first load — register and dashboard open instantly. A background queue pushes new entries to Supabase whenever the connection is available; a sync badge in the top bar shows green / yellow / red with a manual retry. Because the laptop is the Principal's personal device, the app warns her if unsynced entries sit older than two hours. PWA install is v2 — v1 runs in a normal browser tab.

3. Data models

Four tables. v1 writes all four through the UI: families and students via the Profile dialog, fee_structures via Settings, payments via the register grid and the ⋯ modal. Everything else (Pending column, totals, term-paid green ticks) is derived at read time from these four tables.

3.1 Schema diagram — primary keys, foreign keys, cardinality

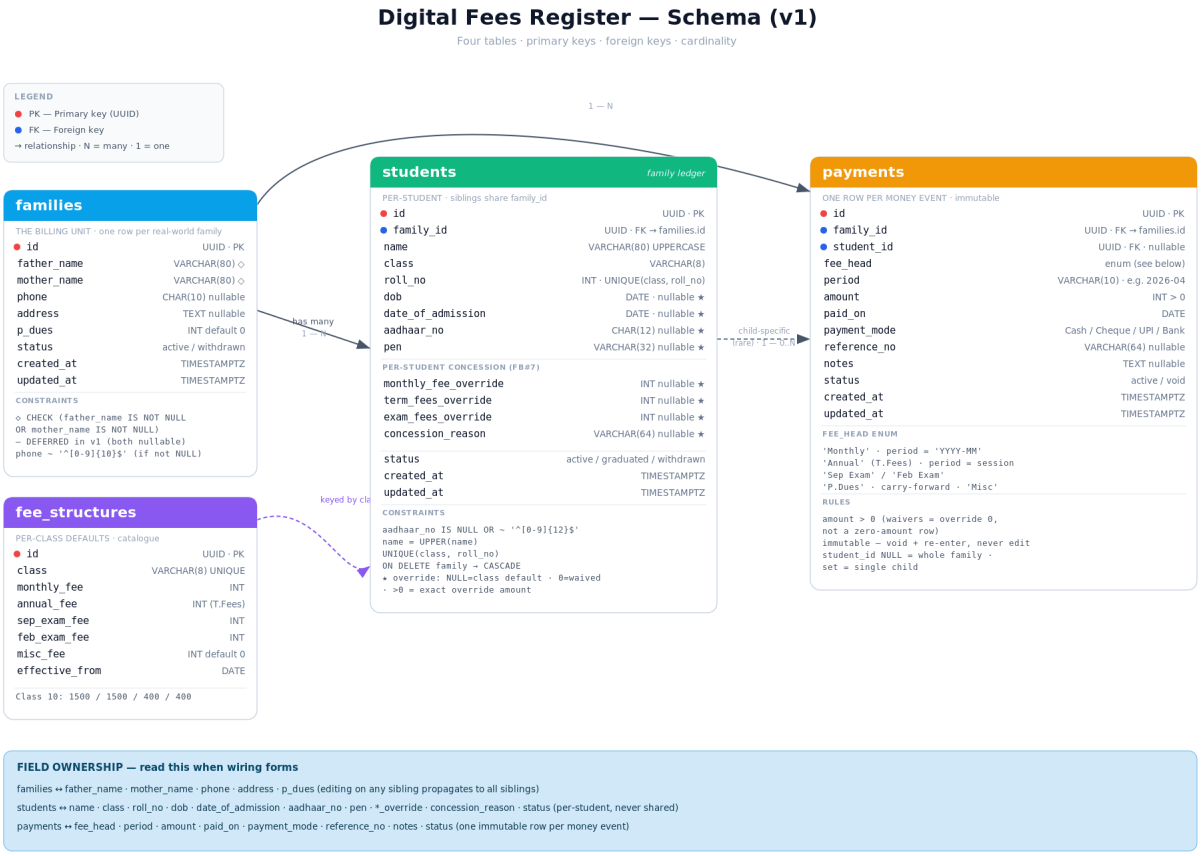

Figure 3.1 — Full schema with PK / FK markers, cardinality, and field-ownership note

3.2 Field ownership — family vs student vs ledger

When wiring forms, the rule is simple: family-shared fields live on the families row and propagate across every sibling; per-student fields live on the students row.

Field	Lives on	Notes
father_name · mother_name · phone · address · p_dues	families	Edit on any sibling propagates to all. At least one of father / mother must be set (CHECK) — DEFERRED in v1; both nullable for now.
name · class · roll_no · dob · date_of_admission · aadhaar_no · pen	students	Per-student, never shared. All name fields are UPPERCASE on both client and server.

Field	Lives on	Notes
monthly_fee_override · term_fees_override · exam_fees_override · concession_reason	**students**	Per-student concession. NULL = use class default · 0 = waived · >0 = exact override.
status (active / graduated / withdrawn)	**students**	Withdrawn / graduated hides the row from future months without deleting history.
status (active / withdrawn)	**families**	Family-level withdraw hides every sibling in one motion.
fee_head · period · amount · paid_on · payment_mode · reference_no · notes · status	**payments**	Ledger only — derived UI states (cell green, .term-paid) read from row existence; never store UI flags.

3.3 Per-table schema

families *one row per real-world family · billing unit · class-agnostic*

Column	Type	Constraint / Default	Notes
id	UUID	PK · DEFAULT gen_random_uuid()	Primary key.
father_name	VARCHAR(80)	nullable ◇	UPPERCASE on insert. CHECK in row constraint.
mother_name	VARCHAR(80)	nullable ◇	UPPERCASE on insert. CHECK in row constraint.
phone	CHAR(10)	nullable, CHECK (phone IS NULL OR phone ~ '^[0-9]{10}$')	India mobile, 10 digits, no country code.
address	TEXT	nullable	Free-text postal address.
p_dues	INT	DEFAULT 0	Prior-session carry-forward dues for the family.
status	TEXT	CHECK IN ('active','withdrawn')	Withdrawn hides every sibling in future months.
created_at	TIMESTAMPTZ	DEFAULT now()	Sync ordering.
updated_at	TIMESTAMPTZ	DEFAULT now()	Sync ordering, updated by trigger.

Row constraint — CHECK (father_name IS NOT NULL OR mother_name IS NOT NULL) · at least one parent must be set. DEFERRED in v1 — both fields nullable; re-enabled in a later iteration.

students *one row per student · siblings share family_id*

Column	Type	Constraint / Default	Notes
id	UUID	PK · DEFAULT gen_random_uuid()	Primary key.
family_id	UUID	NOT NULL · REFERENCES families(id) ON DELETE CASCADE	Sibling grouping. Deleting the family deletes all students.
name	VARCHAR(80)	NOT NULL · CHECK (name = UPPER(name))	UPPERCASE everywhere (FB#1).
class	VARCHAR(8)	NOT NULL	e.g. '10', '7', 'Nursery'.
roll_no	INT	UNIQUE(class, roll_no)	Per-class roll number. Cannot collide within a class.
dob	DATE	nullable ★	Date of birth (FB#2).
date_of_admission	DATE	nullable · DEFAULT CURRENT_DATE ★	Date the student joined (FB#2).
aadhaar_no	CHAR(12)	nullable · CHECK (aadhaar_no IS NULL OR aadhaar_no ~ '^[0-9]{12}$') ★	Stored raw; displayed XXXX XXXX XXXX (FB#3).
pen	VARCHAR(32)	nullable ★	Permanent Education Number. No format validation in v1 (FB#3).
monthly_fee_override	INT	nullable ★	NULL = class default · 0 = waived · >0 = exact override (FB#7).
term_fees_override	INT	nullable ★	Same semantics. Applies to the annual T.Fees head.
exam_fees_override	INT	nullable ★	Same semantics. Covers Sep + Feb exam (v1 — single toggle).
concession_reason	VARCHAR(64)	nullable ★	Free-text reason for audit (FB#7).
status	TEXT	CHECK IN ('active','graduated','withdrawn')	Soft delete and graduation handled here.
created_at	TIMESTAMPTZ	DEFAULT now()	Sync ordering.

Column	Type	Constraint / Default	Notes
updated_at	TIMESTAMPTZ	DEFAULT now()	Sync ordering.

Override semantics (applies to all three *_override columns):

- **NULL** → fall back to the class default in fee_structures.
- **0** → head is waived for this student (concession). Contributes 0 to Pending.
- **>0** → use exact override amount for this student instead of the class default.

fee_structures *per-class fee catalogue · edited via Settings*

Column	Type	Constraint / Default	Notes
id	UUID	PK · DEFAULT gen_random_uuid()	Primary key.
class	VARCHAR(8)	UNIQUE	One row per class.
monthly_fee	INT	NOT NULL	Class default per-month tuition.
annual_fee	INT	NOT NULL	T.Fees (term/annual) charged once per session.
sep_exam_fee	INT	NOT NULL	September exam fee.
feb_exam_fee	INT	NOT NULL	February (pre-board) exam fee.
misc_fee	INT	DEFAULT 0	Miscellaneous bucket; rarely used in v1.
effective_from	DATE	NOT NULL	When this structure took effect (session start).
created_at	TIMESTAMPTZ	DEFAULT now()	
updated_at	TIMESTAMPTZ	DEFAULT now()	

Reference for v1 — Class 10: ₹1,500 monthly · ₹1,500 annual (T.Fees) · ₹400 Sep exam · ₹400 Feb exam → ₹20,300 per student per year.

payments *one row per money event · immutable ledger*

Column	Type	Constraint / Default	Notes
id	UUID	PK · DEFAULT gen_random_uuid()	Primary key.
family_id	UUID	NOT NULL · REFERENCES families(id)	Always set. Every payment belongs to a family.

Column	Type	Constraint / Default	Notes
student_id	UUID	nullable · REFERENCES students(id)	NULL = whole-family payment (99% case). Set = paid for a single child (rare).
fee_head	TEXT	CHECK IN ('Monthly','Annual','Sep Exam','Feb Exam','P.Dues','Misc')	Distinguishes monthly cells from term / exam / carry-forward / misc.
period	VARCHAR(10)	nullable (e.g. '2026-04' for monthly, '2026-27' for annual)	NULL for one-shot heads.
amount	INT	CHECK (amount > 0)	Whole rupees. Never ₹0 — waivers are represented by overrides.
paid_on	DATE	NOT NULL · DEFAULT CURRENT_DATE	When the money was received.
payment_mode	TEXT	CHECK IN ('Cash','Cheque','UPI','Bank Transfer')	Preset chips in the ⋯ modal.
reference_no	VARCHAR(64)	nullable	Cheque / UTR / UPI reference.
notes	TEXT	nullable	Free-text custom note (e.g. “discount approved by Trustee”).
status	TEXT	CHECK IN ('active','void') · DEFAULT 'active'	Void = soft delete; row is preserved for audit. Never edit in place.
created_at	TIMESTAMPTZ	DEFAULT now()	
updated_at	TIMESTAMPTZ	DEFAULT now()	

3.4 Indexes worth creating on day 1

- students(class) — register grid scopes by class on every render.
- students(family_id) — sibling chip rendering.
- payments(family_id, period) — Pending calculation, every cell render.
- payments(family_id, fee_head, period) — Term-paid (Annual / Sep Exam / Feb Exam) existence lookups.
- payments(paid_on) — Today / MTD totals and the Total Transactions screen.

3.5 What is NOT stored (derived at read time)

- **Cell paid/unpaid colour** — derived from whether a payments row exists with matching family_id + fee_head + period.
- **Pending column** — recomputed per row per render (see §4).
- **Today / MTD / Outstanding totals** — aggregated from payments at query time; never written to a totals table.
- **.term-paid green tick** — driven by existence of a payments row with fee_head='Annual' (or 'Sep Exam' / 'Feb Exam') for the family + session year.

4. Validation rules & business logic

4.1 Student Profile — order of checks on Save

Each failure raises a toast and stops the save. Order is intentional: cheapest checks first.

1. Name not empty.
2. At least one of father or mother is set. — SUSPENDED in v1 (both optional); returns in a later iteration.
3. dob is not empty (DATE). — SUSPENDED in v1 (DOB optional).
4. date_of_admission is not empty (DATE). — SUSPENDED in v1 (optional; still defaults to today).
5. phone.length === 10 (digit-only) — validated only when a phone is entered.
6. aadhaar.length === 12 (digit-only) — validated only when an Aadhaar is entered.

PEN is optional, no validation. All name inputs auto-uppercase on the client AND are normalised to uppercase on the server (FB#1). Aadhaar is displayed as XXXX XXXX XXXX but persisted as a raw 12-digit string.

4.2 Pending column calculation

Per family row in the active register class. Never stored — recomputed on render.

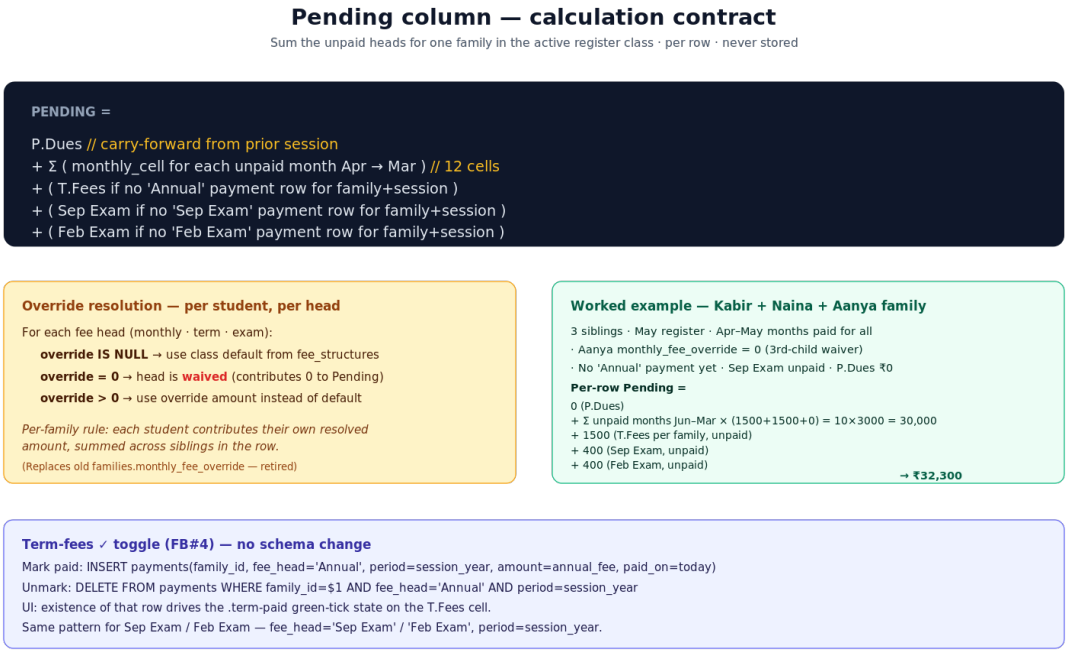

Figure 4.1 — Pending = P.Dues + Σ unpaid month cells + unpaid term / exam heads − waivers

4.3 Term-fees ✓ toggle — no schema change

The T.Fees ✓ (and the September / February exam toggles) re-use the existing payments table. There is no separate flag column.

Mark paid:

```
INSERT INTO payments (family_id, fee_head, period, amount, paid_on, payment_mode,
status)
VALUES ($1, 'Annual', '2026-27', $annual_fee, CURRENT_DATE, 'Cash', 'active');
```

Unmark (toggle off):

```
DELETE FROM payments WHERE family_id=$1 AND fee_head='Annual' AND
period='2026-27';
```

Existence of that row drives the .term-paid green-tick state on the T.Fees cell. Same pattern for Sep Exam / Feb Exam.

4.4 Sibling auto-link and propagation

Every student record is linked to its families row via family_id. Family-shared fields live on families; everything else on students. The user-facing implication is that adding a sibling pre-fills the parent contact section of the new sibling's profile — the Principal doesn't re-type the same data three times.

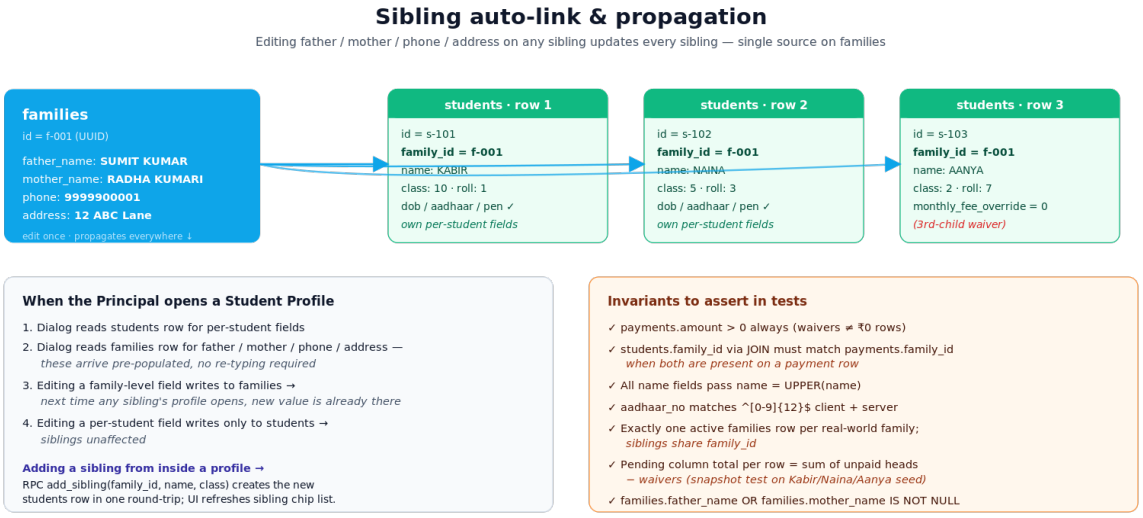

Figure 4.2 — Edit father / mother / phone / address once · all sibling profiles see the new value

Behaviour contract

1. When the Principal saves a Student Profile that lists siblings, the system upserts students rows for those siblings with the same family_id. Their dob / aadhaar / pen / overrides start empty.
2. When the Principal opens any sibling's profile (via Register row ✏ or via a clicked sibling-chip → new class → ✏), the profile dialog reads the students row for per-student fields and the families row for father / mother / phone / address — these arrive pre-populated.
3. Editing father_name / mother_name / phone / address writes to families → next time any sibling's profile is opened, the new value is already there.
4. Editing student-specific fields writes only to students; siblings are not affected.

Recommended Supabase pattern: a single RPC `add_sibling(family_id, name, class)` creates the new students row in one round-trip; the UI then refreshes the sibling-chip list from `students.family_id`. The HTML mockup currently uses a flat FAMILY_PROFILES map and does NOT implement the propagation — this is one of the things the v1 build needs to actually wire.

4.5 Total Transactions (Screen 6) — implementation contract

- Route: /transactions (or in-app #screen-total-tx).
- Data source: full payments table, paginated server-side (50 per page).
- Filters compose: window-chip (preset date range) + class + fee head + student-name search.
- KPI calculations (per active filter window):
  - **Collected =** SUM(payments.amount) WHERE paid_at BETWEEN window_start AND window_end.
  - **Expected =** Σ over active students × Σ fee heads applicable to window, with each student's overrides applied.
  - **Outstanding =** Expected − Collected, floored at 0.
- Realtime subscription on payments so the table and KPIs update live.
- Outstanding sub-line MUST read "In window · not lifetime" so the Principal doesn't misread per-window outstanding as the school's total dues.

5. Scope — v1

v1 is a digital register, nothing more. The Principal records every Class 10 fee payment in the app instead of on paper. That single workflow is what v1 exists to validate.

The Principal is the sole user. Parents keep receiving the school's existing paper Fee Book slip in v1 — the parent experience is unchanged. v1 succeeds when the Principal records payments in the app every day and the paper register is retired.

In v1

- Single Admin login via Supabase Auth (the Principal).
- Class 10 only — about 20 families. Other classes appear as placeholders.
- Register-mirror grid: Roll · Name · P.Dues · T.Fees · Monthly · Apr → Mar (with Sep Exam and Feb Exam inline) · Pending. Five sticky-left columns and a sticky-right Pending column.
- Family rows with two-line names: primary student + sibling chips. Same family appears in every class register it has students in.
- Sibling chips are clickable → jump to the sibling's class register, highlight the row (case-insensitive name match).
- Click a month-cell → inline input pre-filled with expected fee → Enter saves and turns the cell green. The 90% path.
- T.Fees ✓ toggle injected into the T.Fees cell — one-tap mark-paid (FB#4). Re-uses the payments table.
- ⋯ escape-hatch modal for partial / exam / back-fill / void: month chips (multi-month for bulk payments) · editable Expected (apply concession) · numeric Amount · date picker · paid-in-cash / cheque / UPI / bank-transfer preset chips · per-child selector (Whole family default).
- ✏ Student profile modal: name · class · roll · DOB · date of admission · 12-digit Aadhaar · PEN (optional) · 10-digit phone · father / mother (at least one required) · address · dynamic sibling list with name + class · per-student concession (monthly / term / exam overrides + reason).
- Adding a new family opens the profile dialog in draft mode; cancelling rolls back the placeholder row.
- Sibling auto-link: family-shared fields (father / mother / phone / address) auto-populate when opening any sibling's profile.
- Settings → Edit fee structure: per-class form for monthly, annual, Sep exam, Feb exam, misc fees, with live total per year. Saves to fee_structures.
- Row delete and withdraw: hover row → × → reason popover (Typo = hard delete with 8-second undo, Withdrawn = soft delete that grays out future months while preserving payment history).
- Today and MTD totals on the register top bar (current-class scope) and on the Dashboard (school-wide).
- Mobile-friendly Dashboard: 3 key metric cards (Today, MTD, Total pending) and a Students-with-pending-dues table with per-row Notify (WhatsApp v2-stub) and a Notify-all megaphone. Last 20 Entries with Student column + Date column + "View all transactions" link.
- Total Transactions screen (Screen 6) — window-chip filter, class / fee-head / search filters, KPI tiles (Collected · Expected · Outstanding) and a paginated transactions table.
- Offline-resilient writes via Dexie + IndexedDB. Background sync to Supabase.

- Numeric-only inputs everywhere amounts are entered. Phone hard-capped at 10 digits. Aadhaar hard-capped at 12 digits.
- All names UPPERCASE on client and server (FB#1). Aadhaar displayed XXXX XXXX XXXX, stored raw.
- Per-student concession via the three *_override columns on students (FB#7). Replaces the older family-level monthly_fee_override.
- Consistent top bar across all 7 screens — Settings · Notifications · Sign out.

Not in v1

Everything below moves to v2 so the core workflow can ship and be validated first:

- Receipt PDF generation and delivery via WhatsApp / Print / Email.
- Online payments (Razorpay UPI) and auto-reconciliation.
- Custom anchor column types — anchor_columns + family_anchor_values join table.
- Cross-class sibling auto-link with fuzzy matching (v1 uses exact UPPERCASE first-name match).
- Clerk and Viewer roles with a users / role table and audit columns.
- Year-end roll-over (uses students.status = ‘graduated’ at session boundary).
- Concession audit trail (who applied it, when, why beyond the free-text reason).
- Defaulter list with bulk WhatsApp reminders.
- Excel / PDF report exports, dashboard analytics, payment-mode split.
- PWA install for a true offline-first experience.
- Parent self-service login — read-only access to their own child’s fee history (V3).
- Split exam overrides (Sep vs Feb as separate toggles) — v1 covers both with a single exam_fees_override.

6. Scope — v2

Once v1 is in daily use and the paper register is retired, v2 picks up in roughly this order:

- Receipt PDF generation — clean layout, sequential receipt number, fee-head split.
- Receipt delivery — WhatsApp (wa.me) first, then Email and browser Print.
- All remaining classes loaded into fee_structures and the register grid; same family rendered in each class register where it has a student.
- Custom anchor column types (e.g., per-class Activity / Lab / Transport fees) backed by anchor_columns + family_anchor_values.
- Cross-class sibling auto-link with fuzzy matching (Levenshtein ≤ 2) and student-ID-based two-way linking.
- UPI payments and auto-reconciliation via Razorpay.
- Defaulter list with bulk WhatsApp reminders driven by the Pending column.
- Clerk and Viewer roles, backed by a users / role table and per-row audit columns.
- Year-end roll-over — flip students.status, create the new session’s records, carry over P.Dues automatically.
- Concession audit trail — who applied it, when, why; plus a structured concession_type enum.

- Dashboard analytics — class-wise breakdown, payment-mode split, recent-payments full list, charts.
- Excel and PDF report exports.
- PWA install for a true offline-first experience.
- Parent self-service login — read-only access to their own child’s fee history (V3).

7. Open questions deferred to Principal follow-up

Kept in sync with feedback_session_1.md Open Questions. Each item ships in v1 with the recommended default; revisit once the Principal has the live app in hand for a week.

Q1 — First-term-fees month for mid-session admissions — April or DoA month? *Default: April (T.Fees always anchored to session start).*

Q2 — Full filter criteria scope for Screen 6. *Recommend Class + Month + Fee head + Student-name search.*

Q3 — Date format — short on dashboard, long on transactions. *DD MMM on Last 20 Entries, DD MMM YYYY on Screen 6 (recommended).*

Q4 — Session year bounds — April → March (Indian academic standard) for "This year" chip. *Lock as Apr → Mar.*

Q5 — Mid-month window edges in Expected calc — full-month vs prorate. *Recommend full-month for v1.*

Q6 — Auto-detect concession on 3rd sibling vs fully manual. *Recommend manual for v1; revisit when data shows the false-positive rate.*

Q7 — Exam override as single toggle (covers Sep + Feb) vs split. *Recommend single for v1 (one exam_fees_override column).*

Q8 — Concession policy wording on Fee Structure info card. *Open: '3rd child onwards free' vs 'case-by-case'.*

8. Build-time invariants and test seeds

Assert these in unit / integration tests. Each one was learned the hard way during the mockup iterations.

1. **payments.amount > 0 always.** *Concession waivers are represented by *_override = 0, not by a ₹0 payment row.*
2. **payments.family_id matches students.family_id when student_id is set.** *JOIN through student_id must yield the same family_id; otherwise the payment is mis-attributed.*
3. **Pending column total per row = Σ unpaid heads − waivers.** *Snapshot-test against the Kabir / Naina / Aanya example from the mockup.*
4. **students.name, families.father_name, families.mother_name pass name = UPPER(name).** *Server-side normalisation runs before insert/update.*
5. **Aadhaar regex ^[0-9]{12}$ enforced both client and server side.** *Display formatting (XXXX XXXX XXXX) is presentation-only.*
6. **Exactly one active families row per real-world family; siblings share family_id.** *Cross-class sibling fuzzy-match is v2; v1 trusts the Principal's manual link.*
7. **Term-paid existence: at most one active payments row per (family_id, fee_head, period) for Annual / Sep Exam / Feb Exam.** *Enforced by a partial UNIQUE index on (family_id, fee_head, period) WHERE status = 'active' AND fee_head IN ('Annual', 'Sep Exam', 'Feb Exam').*

Test seed — the canonical family

Use this 3-sibling family as the seed for the Pending snapshot test and the sibling-propagation integration test.

family_id	Student	Class	Roll	monthly_fee_override	concession_reason
f-001	KABIR	10	1	NULL	—
f-001	NAINA	5	3	NULL	—
f-001	AANYA	2	7	0	3rd-child waiver (Principal-approved)

Parent contact rows on families: father SUMIT KUMAR, mother RADHA KUMARI, phone 9999900001, address 12 ABC Lane. Editing any of these on Kabir's profile must propagate to Naina's and Aanya's profiles immediately.

9. Cost

The whole point of v1 is that it ships at ₹0 ongoing. v2 stays cheap if we resist the urge to wire a full payment gateway. The two tables below explain what the school actually pays today, what it would pay once SMS reminders and UPI payment links are added, and how to keep that number minimum.

9.1 v1 — what we are shipping now

Every layer of the v1 stack runs on a free tier. The only spend is a small one-time API budget for Claude Code while the engineer is building.

Line item	Provider · plan	Cost
Hosting	Vercel Hobby	₹0 / month
Database + Auth	Supabase Free	₹0 / month (500 MB DB, 50K MAU — ample for a 250-student school)
Code	GitHub private repo	₹0 / month
Domain	vercel.app subdomain	₹0 (custom domain optional: ~₹600–1,000 / year)
Build-time Claude Code API	Anthropic console	~₹1,000–2,500 one-time (entire 7-day build)
Principal usage	Browser on her laptop / phone	₹0

v1 ongoing cost: ₹0 / month.

v1 one-time build cost: roughly ₹1,000–2,500 in Anthropic API spend for the seven-day Claude Code build. Single payment, never recurs.

9.2 v2 — once SMS reminders and UPI payment links are added

v2 is where the principal can text or WhatsApp parents a reminder that contains a one-tap UPI payment link. There are two ways to do this; the cost difference is dramatic.

Path A — Minimum cost (recommended)

Send the reminder via SMS (or WhatsApp click-to-chat link). The link is a direct UPI deep-link (e.g. upi://pay?pa=school@bank&am=1500) that opens the parent's UPI app pre-filled with the amount. The parent pays directly from their bank account to the school's UPI VPA — no payment gateway in the middle, no transaction fee.

Line item	Provider · plan	Cost
SMS reminders	MSG91 / Fast2SMS bulk	~₹0.18 / SMS · ~6,000 SMS / year · ≈ ₹1,100 / year
WhatsApp click-to-chat	wa.me link (no API)	₹0 — parent receives a pre-filled message they tap to send

Line item	Provider · plan	Cost
UPI payment	Direct VPA deep-link	₹0 — money goes straight to school's bank, no gateway
Reconciliation	Manual or bank CSV	₹0 — Principal pastes / imports bank statement weekly
Hosting + DB	Same free tiers as v1	₹0 / month

Path A total cost: roughly ₹100 / month (~₹1,100 / year). Scales linearly with the number of SMS sent.

Path B — Full payment gateway (only if Path A reconciliation becomes painful)

Replaces the direct VPA link with a Razorpay (or Cashfree / PhonePe Business) hosted payment page. The advantage is instant payment confirmation pushed via webhook — the app updates the register without the Principal manually reconciling. The cost is the gateway fee.

Line item	Provider · plan	Cost
Everything in Path A	—	~₹100 / month
Razorpay UPI gateway	Standard rate	2% per UPI transaction (negotiable for education)
Annual fee volume	~₹50L collected / year	≈ ₹1,00,000 / year at 2%
WhatsApp Business API	Cloud API	Free up to 1,000 conversations / month

Path B total cost: roughly ₹8,500 / month (~₹1L / year, dominated by gateway fees). Worth it only if automation removes meaningful Principal time.

9.3 Recommendation and ROI framing

Pick Path A. The cash-flow improvement that v1 already produces (~₹2,900 / month at the 250-student scale per §D2b in Mockup_User_Functionality.md) more than covers Path A's ₹100 / month SMS bill, with the rest accruing as net working-capital benefit to the school.

- Path A net economics: +₹2,800 / month after costs.
- Path B net economics: −₹5,600 / month — gateway fees eat the cash-flow win unless automation savings are very large.
- If the school grows beyond ~250 students or onboards a second campus, revisit Path B with negotiated Razorpay education-sector rates (often well below 2%).

The principle behind this whole stack — including the choice of Supabase and Vercel free tiers over self-hosted infrastructure — is that the product has to pay for itself within the first three months of operation. v1 hits that bar trivially at ₹0 cost. v2 Path A keeps the same property. Path B does not.

— end of technical specification —